Information Processing and Management 62 (2025) 104013



Contents lists available at ScienceDirect

Information Processing and Management

journal homepage: www.elsevier.com/locate/ipm





Unsupervised microservice system anomaly detection via contrastive multi-modal representation clustering

Peipeng Wang, Xiuguo Zhang *, Yutian Chen, Zhiying Cao

Information Science and Technology College, Dalian Maritime University, Dalian, Liaoning, China

ARTICLE INFO

Keywords:
Multi-modal
Transformer
Anomaly detection
Contrastive loss

ABSTRACT

Anomaly detection in microservice systems is crucial for ensuring system stability and reliability. Existing methods rely solely on a single type of monitoring data (e.g., metrics or logs) with either partial or full labels, which miss a large number of anomalies and is costly to manually tag. Therefore, we propose an unsupervised Microservice system Anomaly Detection method via Contrastive Multi-modal representation Clustering (MAD-CMC) to tackle these issues. MAD-CMC first adopts a hierarchical architecture to simultaneously explore the spatial–temporal correlation of metrics and log context information. Next, to facilitate metrics-logs interaction, MAD-CMC introduces a cross-modal Transformer, which outputs multi-modal representation for clustering. During the clustering process, we design a multi-grained contrastive learning approach. Benefiting from the clustering results, MAD-CMC bring intra-cluster representation closer while pushing inter-cluster representation farther away at both inter- and intra-modality aspect. Considering that normal samples are simpler and far more numerous than abnormal sample, we propose a dynamic weighting formula, and apply it to contrastive loss to improve the model's discrimination ability. Sufficient experiments on public dataset show that MAD-CMC outperforms state-of-the-art methods.

1. Introduction

In recent years, microservice systems have grown in size and complexity, with different services showing correlation with each other. Once an anomaly occurs in a specific service, it may lead to the entire system collapsing, resulting in irreversible economic losses for enterprises and users (Zhou et al., 2021). In order to enhance the competitiveness of enterprises, service providers will strive to ensure the reliability and stability of their services. In this process, accurately detecting the abnormal behavior of the system is the most basic and critical part, so automated anomaly detection is crucial (He et al., 2021).

During the operation of microservice systems, the execution status of the system will be recorded in various ways, including metrics, logs and traces, which are crucial to maintaining the stability of the system. In particular, metrics and logs can be collected in real time by operators to determine whether the system is abnormal. Metrics are employed to measure actual system performance, such as response time, throughput, error rate, etc., which belong to time series. Logs are recorded information predefined by programmers to describe specific service behaviors, which belong to semi-structured text. Recent studies have primarily focused on analyzing the temporal features of metrics (Zhao, Han, Li, Su and Zhou, 2023) or the semantic features of logs (Wang, Zhang, Cao, Xu, & Li, 2024) to detect anomalies in the system. However, relying solely on a single type of monitoring data is often insufficient to represent the abnormal behavior of the system (Zhao et al., 2021).

* Corresponding author.
E-mail addresses: wpp7@dlmu.edu.cn (P. Wang), zhangxg@dlmu.edu.cn (X. Zhang), chenyutian@dlmu.edu.com (Y. Chen), czysophy@dlmu.edu.cn (Z. Cao).

https://doi.org/10.1016/j.ipm.2024.104013
Received 2 July 2024; Received in revised form 28 November 2024; Accepted 29 November 2024
Available online 10 December 2024
0306-4573/© 2024 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Some studies introduce metrics and logs in traces to comprehensively analyze the data patterns of microservice systems by utilizing three types of monitoring data simultaneously (Huang, Yang, Yu, Li, & Zheng, 2024; Lee, Yang, Chen, Su and Lyu, 2023; Zhang et al., 2023). However, trace collection requires pre-instrumentation, which is not friendly to already deployed microservices (Liu et al., 2024). Moreover, while three-modal data offers more extensive system information, the generation of traces typically involves upstream and downstream service calls, resulting in data collection delays that hinder timely anomaly detection, and processing trace is more complex compared to metrics and logs. Therefore, when operators need to conduct a system-based anomaly detection, they can combine the three-modal data. However, to enhance the efficiency of the system, this study utilizes two-modal data consisting of metrics and logs to detect system anomalies quickly.

Two-modal anomaly detection approaches mainly utilize feature splicing or cross-modal interaction to generate multimodal features. Among them, the splicing-based methods inevitably ignores the representation differences between different modalities. The interaction-based methods extract metric time features and log semantics respectively, and employs the attention mechanism to interact to obtain multimodal representation, Lee, Yang, Chen, Su, Yang et al. (2023) which improves the anomaly detection effect.

However, these interaction-based methods still ignore the statistical feature of logs and the spatial correlation of metrics, resulting in limited anomaly detection performance. The log statistical feature refers to a quantitative relationship in the operation process of the microservice system. Such as the opening of a file corresponding to the closing of a file, and the starting of a process corresponding to the termination of a process (Meng et al., 2019). Analyzing this statistical relationship can help to identify quantitative anomalies. Metric spatial correlation refers to the dependence between different variables. For example, longer response times may be associated with lower throughput, and the error rate of system may increase under higher loads. Spatial correlation, as a key means of improving performance in time series research, has been neglected in multimodal anomaly detection (Shi et al., 2023). These multifaceted and heterogeneous data features necessitate that the model can flexibly handle a variety of information. In this paper, we adopt a hierarchical architecture to extract advanced patterns of multimodal data (i.e., semantic and statistical features of logs, spatio-temporal correlation of metrics), and introduce a cross-modal Transformer model to perform deep cross modal interactions to compensate for the representation differences between modalities.

Furthermore, supervised learning has been widely utilized for anomaly detection (Lee, Yang, Chen, Su, Yang et al., 2023). Although this approach can achieve high accuracy, annotating multimodal data rich expert knowledge and is time-consuming, especially for the launch period of new services, making it difficult to obtain sufficient labeled data. To this end, some studies utilize normal monitoring data for reconstruction training (Liu et al., 2024). While this approach avoids the reliance on labels, it is impractical to obtain completely normal monitoring data. The reason is that the complexity and dynamics of the microservice systems often lead to noise, emergencies or hidden anomalies in the actual operation process, and the fusion of multimodal data will amplify these abnormal manifestations, resulting in inaccurate normal representation. In this paper, we design a clustering method based on multimodal representations to distinguish normal, abnormal behaviors of a system in an unsupervised manner and utilize contrastive learning to enhance the discriminative ability of the model.

We propose MAD-CMC, a unsupervised microservice system anomaly detection method via contrastive multi-modal representation clustering. It includes the following four modules: (1) For metrics, we use Pearson coefficients to describe the relationship between different variables and construct the metric feature graph. VGAE (Kipf & Welling, 2016) is utilized to reconstruct feature graphs so that its encoder can extract the metric hidden state containing spatial dependencies. (2) For logs, we combine BERT (Devlin, Chang, Lee, & Kristina, 2019) and SVD (Ge, Weng, & Yang, 2014) to obtain compact log semantics. Subsequently, we extract statistical features and concatenated them with semantics. Next, an AE (Srivastava, Mansimov, & Salakhutdinov, 2015) is trained to reconstruct the semantic and statistical features so that its encoder can fully extract the log context information. (3) The cross-modal Transformer model (Tsai et al., 2019) is utilized to analyze temporal relationships between metrics and logs and perform cross-modal interactions, obtaining multi-modal representation. (4) Next, the K-means algorithm (Yang et al., 2023) is utilized for clustering. To leverage the advantages of contrastive learning in deep clustering (Hu et al., 2023), we design a multi-grained contrastive learning method to optimize multi-modal representations from both intra- and inter-modal aspects. It is worth noting that in order to enhance the discriminative ability of the model, we introduce a dynamic weighting formula for the contrastive loss by adjusting the weights of difficult and simple samples in unbalanced datasets. Therefore, multimodal representation and clustering interact and jointly evolve in an unsupervised manner in MAD-CMC. Finally, each cluster is converted into anomaly detection results.

The main contributions of this paper are as follows:

- We propose an anomaly detection method for microservice systems, MAD-CMC. It jointly analyzes metrics and logs in an unsupervised manner within a framework. To the best of our knowledge, this paper is the first study to perform contrastive deep clustering for anomaly detection in multi-modal monitoring data.
- We explore multimodal data features from multiple perspectives. AE is employed to extract both semantic and statistical features from logs, VGAE is used to analyze spatial dependencies between variables, and a cross-modal Transformer model is utilized to further learn their temporal features and facilitate cross-modal interactions.
- We propose a multi-grained contrastive learning approach, guided by clustering pseudo labels, to optimize multi-modal representations from both intra- and inter-modal aspects.
- A dynamic weighting formula is designed to assist in contrastive learning. It flexibly reduces the weight of normal simple samples and increases the weight of abnormal ones, thereby enhancing the discriminative ability of the model.

2. Related work

2.1. Single modal data

Log. The entire log anomaly detection process can be divided into three steps: log parsing, feature extraction, and anomaly detection. First, by utilizing various log parsers such as Drain (He, Zhu, Zheng, & Lyu, 2017), IPLoM (Makanju, Zincir-Heywood, & Milios, 2009), LogMin (Hamooni et al., 2016), etc., to convert semi-structured log messages into structured log events, useless information is removed, noise interference is avoided, and a foundation is laid for subsequent anomaly detection.

Subsequently, log sequences are constructed based on the parsed log events, which are then encoded into feature vectors using various feature extraction methods. Mainly categorized into index-based and semantic-based approaches. DeepLog (Du, Li, Zheng, & Srikumar, 2017) treated log events as indexes and assigns a unique ID to each event. However, this approach ignores the log semantics, which leads to poor robustness of the model. Therefore, word embedding techniques are increasingly utilized to capture semantic information. LogRobust (Zhang et al., 2019) combined Fast-text and tf-idf algorithms to achieve this. MLog (Fu, Liang, & Xu, 2023) combined transformer encoder and Inverse Document Frequency (IDF) to extract log semantics. SwissLog (Li, Chen, Jing, He, & Yu, 2023) employed the BERT model to obtain sentence vectors from log events. Despite analyzing the log semantics from various perspectives, they overlooked the statistical aspect of log events.

Finally, in order to analyze log features and detect anomalies, DeepSysLog (Ding, Trajcevski, Scheuermann, Wang and Keogh, 2022) used LSTM and MLP to detect anomalous sequences from normal log sequences. QLLog (Duan, Ying, Yuan, Cheng, & Yin, 2021) considered log anomaly detection as a sequential decision problem, the Q-learning algorithm was utilized to identify abnormal log events and ranked the severity level of abnormal events. To better represent the relationships of log sequences, GAELog (Xie & Yang, 2024) modeled log sequences as knowledge graphs and event graphs, and employed AE for anomaly detection. LogAnomaly (Meng et al., 2019) was firstly combined statistical relationships of logs to analyze anomalies, which is worth learning from. Yan, Luo, and Shao (2023) further examined the temporal relationship between log events and constructed a temporal event graph for anomaly detection based on generative adversarial networks.

Metric. Metrics are typically represented as time series data, existing literature focuses on deep learning models. For example, Hundman (Hundman, Constantinou, Laporte, Colwell, & Soderstrom, 2018) utilized LSTM to analyze the time series and determine whether an anomaly has occurred by predicting new-coming values. MM (Zhao, Han et al., 2023) designed a semi-random connection RNN to capture metrics's feature to the most extent. OmniAnomaly (Su et al., 2019) utilized the GRU model to analyze temporal dependencies and the VAE-based reconstruction model to calculate anomaly scores. Anomaly-Transformer (Xu, Wu, Wang, & Long, 2022) used Transformer to analyze metric and introduce the correlation difference loss function to measure the correlation difference between normal and abnormal series. Whereas in microservice systems, various metrics interact with each other. To address this, MUTANT (Shi et al., 2023) created edges for similar metrics using compositional graphs, learned the graph embeddings using GCN, and reconstructed the original inputs for anomaly detection using the VAE model. MST-GAT (Ding, Sun and Zhao, 2022) adopt a self-learning approach to construct metric graphs. The multimodal-GAT is used to extract spatio-temporal features, with the reconstruction of the VAE and the prediction of the MLP for joint optimization.

While these methods take into account the operational patterns of system from various perspectives, it is important to note that the system information contained in the single data source is limited. Therefore, MAD-CMC models capture log semantics and statistical features while integrating the spatio-temporal correlation of metric to provide a more comprehensive description of system status.

2.2. Multi-modal data

Some studies use three types of monitoring data, namely metrics, logs and traces, to perform tri-modal anomaly detection, which has become a new trend. They mainly introduce metrics and logs in the corresponding services and time periods based on the trace to better analyze the status of the microservice system. For example, Eadro (Lee, Yang, Chen, Su, Lyu, 2023) utilized the trace to construct dependency graph that incorporate log semantics and metric time features. However, this method overlooks the representation difference between metric and log. Therefore, DiagFusion (Zhang et al., 2023) converted logs into index sequences and embeds them into the trace graph, while manually increasing the number of abnormal samples. MSTGAD (Huang et al., 2024) also modeled logs as time series and integrated metrics and logs within the trace graph. Although they avoid the representation differences across modalities, it still requires labeled data for supervised or semi-supervised training. To address this issue, AnoFusion (Zhao, Ma et al., 2023) divided the tri-modal data into time series, a graph Transformer was employed to learn the correlation between different modal, and performed anomaly detection by predicting new-coming values. This approach achieves higher accuracy, but the introduction of trace causes the spatial dependency of metrics as well as the semantic and statistical characteristics of logs to be ignored.

In practical applications, the three-modal anomaly detection approaches can utilize trace to comprehensively analyze the propagation path of anomalies. However, not all systems can easily obtain trace data, and trace collection may increase system load and increase response time. Therefore, in scenarios with rapid response requirements and limited resources, it is a better solution to make full use of metric and logs for two-modal anomaly detection.

In anomaly detection that combines metrics and logs, one approach is to perform feature splicing to obtain multimodal features. For instance, SCWarn (Zhao et al., 2021) modeled logs as time series and utilized LSTMs with the same structure to extract indicators and log features. DAM (Chen, Yan, Yang, Zhang, & Wang, 2022) further divided metrics into business metrics and performance

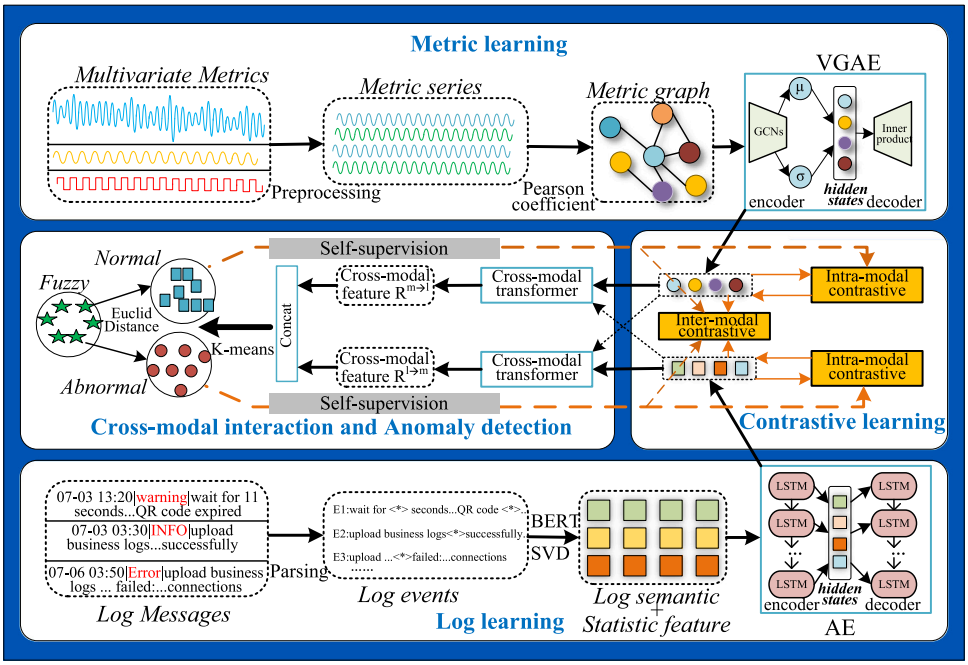


Fig. 1. The framework of MAD-CMC. It consists of four main sections: (1) Metric learning and (2) Log learning module conduct pre-training. VGAE and AE are used to automatically learn various metric embeddings and log contexts, respectively, to obtain the hidden states. (3) Contrastive Learning and (4) Cross-modal interaction and anomaly detection module conduct fine tuning. Taking different modal hidden states as input, cross-modal Transformer is utilized to facilitate metric-log interaction, generating multimodal features for clustering. Benefiting from the clustered pseudo-labels, intra- and inter-modality contrastive learning are performed to optimize multimodal representations.

metrics, obtains multimodal features by splicing the two metrics and log representations. Considering that these methods ignore the representation differences of metrics and los, Hades (Lee, Yang, Chen, Su, Yang et al., 2023) utilized Transformer to analyze log semantics, employed a causal convolutional neural network (CNN) to encode metrics, and implemented an attention mechanism to facilitate cross-modal interactions. To avoid dependence on labels, UAC-AD (Liu et al., 2024) adopted reconstruction-based approaches and introduced contrastive learning and adversarial training to distinguish normal and hard samples. However, due to the complexity and dynamics of microservice systems, there is often noise or hidden anomalies in the actual normal samples. Therefore, MADM-CMC combines contrastive learning with clustering (Xia, Wang, Gao, Yang, & Gao, 2023) for unsupervised joint optimization of multimodal representations and detection results.

3. Method

Problem Definition. Consistent with existing anomaly detection methods (Lee, Yang, Chen, Su, Yang et al., 2023; Liu et al., 2024; Su et al., 2019; Zhang et al., 2019), this paper focuses on service instance-based methods rather than system-based ones. Given multimodal monitoring data, MAD-CMC aims to determine whether each service instance within the system is normal or abnormal. If any instance indicates an anomaly, it classifies the entire system as anomalous. The overall framework is shown in Fig. 1. Taking multimodal monitoring data as input, after the metric and log learning module, the contrastive learning module, and the cross-modal interaction and anomaly detection module, the detection results are output. Formally, we define the monitoring data and label as $\langle X_{1:n}, Y_{1:n}\rangle = \{(X_1, y_1), (X_2, y_2), \ldots, (X_n, y_n)\}$, where $X_i = (X_i^m, X_i^l)$ represent metrics and logs, respectively, and we describe them in detail in Sections 3.1 and 3.2. $y_i \in \{0, 1\}$ denotes the status, 1 is abnormal. Existing studies require a large number of labels to learn a discriminative system status, but obtaining enough anomaly labels is labor-intensive and prone to introduce noise. Therefore, we propose MAD-CMC to train the model in an unsupervised manner. It utilizes a clustering algorithm to directly classify a large amount of monitoring data $X_{1:n}$ into different clusters $\tilde{Y}_{1:I}$, where I is the number of clusters. Next, the clustering centers for normal and abnormal status are determined based on the number of samples and relationships in the clusters. In this way, for new monitoring data X_{n+1}, MAD-CMC can predict the status y_{n+1}.

3.1. Metric learning

In the multimodal data shown in Fig. 2, metrics are multivariate time series, which are continuous observations collected from different sources at equal interval timestamps. Metrics learning mainly consists of the following processes.

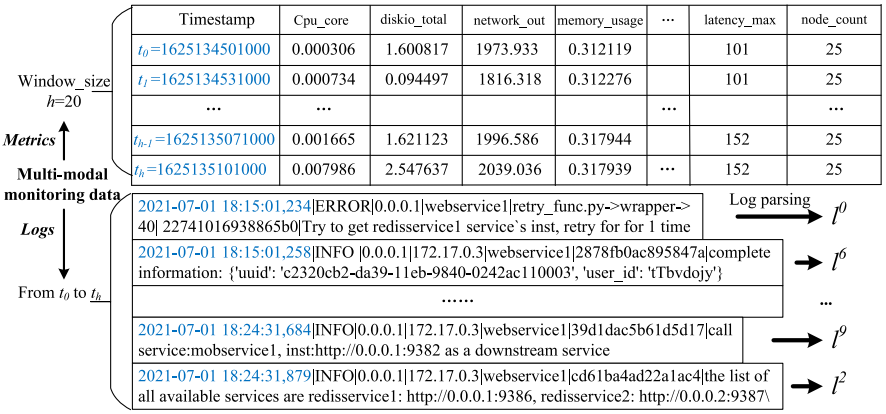


Fig. 2. An example of multimodal data.

Preprocessing. We define metric as $M = \{m_1, m_2, \dots, m_n\}$, where n represents the length of M, $m_t \in \mathbb{R}^k$ is a k dimensional observation vector, i.e. $m_t = \{m_t^1, m_t^2, \dots, m_t^k\}$, k is the number of variables, which may indicate CPU utilization, response time, throughput, etc. To eliminate dimensional differences between metrics, make it easier for the model to learn the relationships between variables, we first normalize metrics by maximum and minimum values (Zhao et al., 2020). Then, we divide the time series by a window of size h, the time window at current time t is $X_t^m = \{m_{t-h+1}, \dots, m_{t-1}, m_t\}$. The original metric series can be transformed into a sequence of time windows $X_{1:n}^m = \{X_1^m, X_2^m, \dots, X_n^m\}$.

Graph structure learning. By combining previous studies (Shi et al., 2023; Su et al., 2019) with our analysis of metrics, we discover that various metric variables in microservices exhibit linear correlations. The Pearson correlation coefficient effectively captures this relationship, providing a value between −1 and 1. This allows operators to intuitively identify which variables have strong correlations, enabling them to make informed decisions swiftly. In other correlation analysis methods, such as Dynamic Time Warping (DTW) and PCMCI (Runge, Nowack, Kretschmer, Flaxman, & Sejdinovic, 2019) algorithms. The DWT is founded on the principles of dynamic programming and is capable of identifying the optimal matching path through nonlinear time alignment. Consequently, it offers significant advantages in addressing issues of time lag and asynchrony between time series data. For example, changes in response time in some cases can affect future values of error rates. But after data preprocessing, we successfully aligned various variables. The PCMCI algorithm is a causal inference method derived from the PC (Peter-Clark) algorithm. It effectively captures causal relationships between variables, especially suitable for high-dimensional nonlinear data, and can also model lag effects. However, we are mainly concerned with the correlation between various variables, not the specific causal relationship. Moreover, given the real-time data flow in microservices, these algorithms are computationally intensive and lack real-time processing capabilities within the hierarchical architecture of MAD-CMC. Therefore, we choose the efficient Pearson coefficient $\rho_{i,j}$ to explore the correlation between variables and clearly construct the metric feature graph $G^m = \{V^m, E^m\}$ for each time windows. Where, node stores a time series representation $X_t^{m_i} = \{m_{t-h+1}^i, \dots, m_{t-1}^i, m_t^i\}$, and edges are used to indicate the relationship between these variables. The calculation is shown in Eq. (1).

$$\rho_{i,j} = \frac{Cov(X_t^{m_i}, X_t^{m_j})}{\sqrt{Var[X_t^{m_i}]Var[X_t^{m_j}]}} \tag{1}$$

where $Cov(X_t^{m_i}, X_t^{m_j})$ represents the covariance, $Var[\cdot]$ is the variance. We compare the size of $\rho_{i,j}$ and select top o most relevant variables to establish an edge. In this way, we obtain the adjacency matrix $A^m \in \mathbb{R}^{k\times k}$ of graph G^m, $A^{m_i,j} = 1$ means that there is an edge between nodes v_i^m and v_j^m. Furthermore, considering the causal and lag relationships among various variables in microservice systems, we will utilize algorithms such as PCMCI to investigate the impact of these relationships on the status of system in future research.

Variational Graph Auto-Encoders (VGAE). In the metric feature graph, different nodes represent different metric variables. As shown in Fig. 3, we employ VGAE to efficiently capture the potential dependency between variables. The VGAE encoder utilizes two layers of GCN to obtain the mean μ and variance σ^2, respectively. The hidden state Z_t^m of the metric is obtained by sampling from the normal distribution $\mathcal{N}(\mu, \sigma^2)$. Then, decoder utilizes hidden state Z_t^m to reconstruct adjacency matrix $\tilde{A}$. After designing the encoder and decoder, the reconstruction loss is shown in Eq. (2).

$$L_{m,t} = -\log p(\hat{A}_t | Z_t^m) + \mathrm{KL}(q(Z_t^m | \hat{A}_t, X_t^m) \parallel p(Z_t^m)) \tag{2}$$

where $p(\hat{A}_t | Z_t^m)$ represents the generation probability of the observed adjacency matrix $\hat{A}_t$ under the condition of hidden state Z_t^m. $p(Z_t^m)$ is prior distribution of Z_t^m. Since $p(\hat{A}_t | Z_t^m)$ is difficult to compute directly, the VGAE encoder utilizes an inference network $q(Z_t^m | \hat{A}_t, X_t^m)$ to approximate the estimate.

VGAE reconstructs the structure and features of all variables during pre-training, allowing its encoder to fully extract the hidden state Z_t^m of metrics that include spatial dependencies.

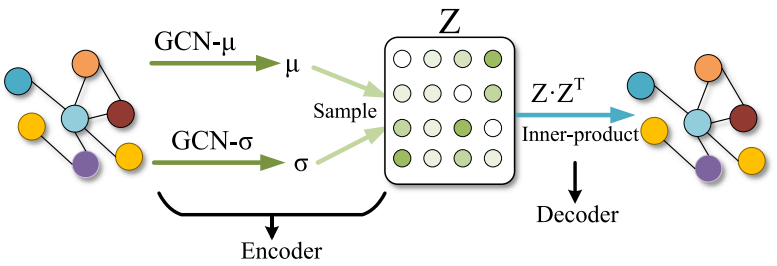


Fig. 3. The structure of VGAE. The encoder utilized two layers of GCN to generate latent variable distribution parameters, namely mean μ and variance σ, the reparameterization technique is used to obtain latent variable features. The decoder reconstructs the adjacency matrix through the inner product and a non-linear function(sigmoid).

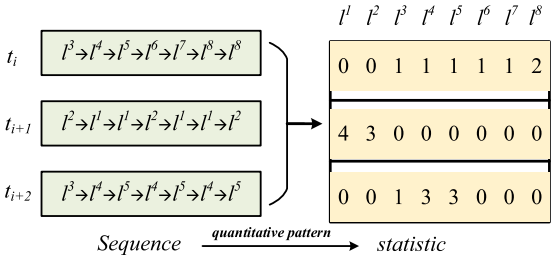


Fig. 4. Log sequence and statistical feature.

3.2. Log learning

Log parsing. In the multimodal data shown in Fig. 2, log messages are unstructured and contain noisy information, which can be transformed into structured log templates through log parsing. Zhu et al. (2019) showed that Drain has higher detection accuracy compared to other parsers. So, MAD-CMC adopts the Drain algorithm to obtain structured log templates. Each template represents a specific log event, and we assign a unique "ID" to each event. We define as $L = \{l^1, l^2, \ldots, l^g\}$, g is the number of log events.

Subsequently, considering the different sampling intervals of logs and metrics, metrics are collected in seconds, while logs are generated in tens or hundreds of entries per second. Therefore, we sequentially arrange the occurrence frequency of event "ID" in order and extract the least frequent u log event, and save their positions in the original log sequences. Ultimately, we represent metrics and logs as a "one-to-many" relationship, and transform logs message into a time window sequence $X^l_{1:n} = \{X^l_1, X^l_2, \ldots, X^l_n\}$ at the same time intervals as the metrics, where $X^l_t = \{l_{t,1}, l_{t,2}, \ldots, l_{t,n1}\}$, $n1$ is the number of log events in the current window. Unlike high-frequency events, lower-frequency log events represent uncommon system states and are more indicative of whether an anomaly is occurring in the system.

Log semantics and statistical feature. To adapt to the continuously updated log statements and get stable log semantics, we employ the BERT model to generate sentence vectors v^1_i, which take the entire log event instead of log words as input to avoid word order loss. Next, considering (1) The statistical feature dimension is low, and it is necessary to balance the deviation between the semantic and statistical feature dimensions. (2) The number of variable in the metric is limited, and it is necessary to avoid information misalignment between high-dimensional log features and low-dimensional metric patterns. So, the SVD algorithm is further employed to convert the original sentence embedding $v^1_i \in \mathbb{R}^{768}$ into the compact representation $v_i \in \mathbb{R}^h$, h is the size of the time window. Then, the log semantic representation $V_t \in \mathbb{R}^{h \times n1}$ in the current time window is obtained by mapping the log sentence vector to the log sequence.

In order to effectively represent the constant relationship during the execution of a microservice program to capture quantitative anomalies. As shown in Fig. 4, MAD-CMC quantifies this relationship by analyzing the statistical feature of log events. Specifically, based on the sampling interval of the metrics, we calculate the frequency of each log event occurring within that interval to obtain the count vector c_i, $i \in [1, g]$, for each event at the current time. We then aggregate the count vectors from different moments within the current time window and derive the statistical features $C_t \in \mathbb{R}^{h \times g}$. Next, we splice semantic and statistical features to obtain log sequence features $E_t = concat(V_t, C_t)$. **AutoEncoder (AE).** In order to adaptively learn the normal and abnormal states of log sequences and flexibly handle different types of log events, MAD-CMC utilizes the LSTM-based AE to reconstruct log sequence features and unsupervised learn log context information. LSTM uses four different gate functions with log sequence features E_t as input to obtain the hidden state Z^l_t of the log sequence.

In the decoding process, the hidden states Z^l_t are used as inputs, and the decoder outputs reconstructed log sequence feature E^r_t. The mean square error is used as the loss function.

$$L_l = \frac{1}{n} \sum_{t=1}^{n} (E_t - E^r_t)^2 \tag{3}$$

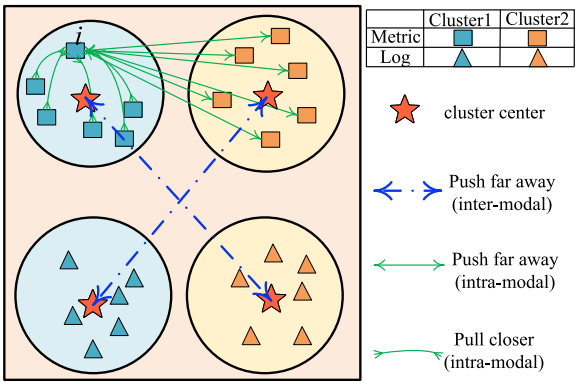


Fig. 5. The illustration of multi-grained contrastive learning. Circles of different colors represent different clusters. MAD-CMC utilizes clustering results as a guide to perform intra-modal and inter-modal contrastive learning, thereby jointly optimizing multimodal representation and clustering results. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

where n is the number of log sequences. Similarly, we reconstruct the semantic and statistical features of normal and abnormal log sequences, so that the pre-trained AE possesses a better reconstruction ability for full-sample log sequences, allowing its encoder to extract the hidden state $Z_t^l \in \mathbb{R}^{g \times d_l}$ of the log sequence context.

3.3. Clustered label-guided contrastive learning

As shown in Fig. 5, to mitigate the reliance on labeled data, we develop a multi-grained contrastive learning method, which utilizes pseudo-labels obtained through clustering as supervision information (Hu, Zhang, Zou, Lou, & Ye, 2024), to capture discriminative features from both intra- and inter-modality aspects.

Intra-modality contrastive learning. Traditional contrastive learning is generally based on InfoNCE (Oord, Li, & Vinyals, 2018) as the loss function, which helps the model learn more useful representations by comparing the similarity between positive and negative samples. The calculation is shown in the Eq. (4).

$$L_{info}^{i} = -\log \frac{e^{sim(x_i, x_i^+)/\gamma}}{e^{sim(x_i, x_i^+)/\gamma} + \sum_{j \neq i} e^{sim(x_i, x_j^-)/\gamma}} \tag{4}$$

where x_i is the embedding of sample i, x_i^+ is positive sample embedding corresponding to i, x_i^- is positive sample embedding corresponding to i. $sim(\cdot)$ represents similarity calculation. γ temperature parameter which used to adjust the similarity scale.

However, using InfoNCE loss directly as intra-modal contrastive loss is inappropriate for two reasons: (1) In general, the system is operating normally, and any anomalies are occurring accidentally. Therefore, a small amount of abnormal data is concealed within a large amount of normal data, and the majority of normal samples are simple data. The infoNCE loss necessitates similarity calculation for each positive and negative sample, which can result in the model being biased towards normal samples and overlooking abnormities. (2) In the monitoring data of microservices systems, occasional fluctuations in metrics or updates in log events may occur. Compared to ordinary monitoring data, these samples may be identified as hard samples by the model. When clustering, normal hard samples may exhibit greater similarity to abnormal samples and lower similarity to normal samples at the center of the cluster. However, InfoNCE treats all samples equally in similarity calculation (Liu et al., 2023), leading to information imbalance and limiting the model's discriminative performance. To solve above problem, benefiting from Focal loss's approach to handling long-tail problems, we design a dynamic weighting formula $S^{(a)}$ based on the sample ratio and similarity between different clusters.

$$S_{i,j}^{(a)} = \begin{cases} [1 - sim(Z_i^{(a)}, Z_j^{(a)})]^{\phi}; i,j \in one - class \\ sim(Z_i^{(a)}, Z_j^{(a)})^{\phi}; i,j \in different - class \end{cases} ; \phi = \lg {}^{N(nor)}/_{N(abnor)} \tag{5}$$

where $a = m$ or l, $(Z_i^{(a)}, Z_j^{(a)})$ represents the ith and jth samples of the same modality. ϕ stands for scaling factor, it is determined based on the proportion of normal and abnormal samples in the classification cluster, usually $1 \leq \phi \leq 4$. It is evident that for samples of the same classes, when the similarity of the sample pairs is low, it indicates that they are hard samples and $S^{(a)}$ is able to adjust the proportion of hard samples upwards while decreasing the proportion of simple samples. In addition, if the proportion of samples of different classes increases, then ϕ increases, resulting in an increase in the decrease rate of simple samples. Finally, we combine $S^{(a)}$ and InfoNCE loss to perform intra-modality contrastive learning in metrics and log hidden states, as shown in Eq. (6).

$$L_{tra}^{(a)} = -\frac{1}{n} \sum_{i=1}^{n} \log \frac{\sum_{j=1}^{M1} e^{S_{i,j}^{(a)} \cdot sim(Z_i^{(a)}, Z_j^{(a)})/\tau}}{\sum_{j=1}^{M1} e^{S_{i,j}^{(a)} \cdot sim(Z_i^{(a)}, Z_j^{(a)})/\tau} + \sum_{k=1}^{n-M1} e^{S_{i,j}^{(a)} \cdot sim(Z_i^{(a)}, Z_k^{(a)})/\tau}} \tag{6}$$

where $M1$ represents the number of samples of the same kind as sample i, which is obtained based on the clustering pseudo-labeling, $i \neq j$ avoids the sample from making similarity calculations with itself.

Inter-modality contrastive learning. When analyzed from a system-wide perspective, metrics and logs work together to represent service status and maintain a complementary relationship. As a result, they exhibit both modality-specific and modality-consistent representations. In the intra-modality contrastive learning described above, we enhance the modality-specific representation by pulling in samples of the same modality within the same cluster while distancing samples from different clusters of the same modality. In inter-modality contrastive learning, we focus on modality-consistent representations. During this process, samples from the same cluster across different modalities can be considered positive pairs, while samples from different clusters of varying modalities can be classified as negative pairs. However, since cluster labels can effectively guide intra-modal contrastive learning, over-pulling positive pairs may lead to an overlap between the modality-consistent representation and the modality-specific representation. Although this issue can be mitigated through regularization, it introduces additional computational overhead. Therefore, inter-modality contrastive learning emphasizes pushing farther away the representation of different modal samples from different clusters to avoid training instability.

First of all, benefitting from the pseudo labels generated by clustering, we calculate the clustering centers of the metrics and logs in the different clusters, serving as the representation centers for specific modalities:

$$cen^{m}_{pos} = avg(Z^{m}_{i}), i = 1, 2, \ldots, m1; cen^{m}_{neg} = avg(Z^{m}_{j}), j = 1, 2, \ldots, n - m1 \tag{7}$$

$$cen^{l}_{pos} = avg(Z^{l}_{i}), i = 1, 2, \ldots, l1; cen^{l}_{neg} = avg(Z^{l}_{j}), j = 1, 2, \ldots, n - l1 \tag{8}$$

where cen^{m}_{pos}, cen^{l}_{pos}, cen^{m}_{neg} and cen^{l}_{neg} are the representation centers of metrics and logs in different clusters, respectively. $avg(\cdot)$ is the mean function, $m1$ and $l1$ represent the number of metrics and log samples in the cluster.

Subsequently, MAD-CMC obtains inter-modality contrastive loss by extending the spatial distance of specific modal clustering centers:

$$L^{(l\to m)}_{ter} = \frac{cen^{l}_{pos} \cdot cen^{m}_{neg}}{\|cen^{l}_{pos}\| \cdot \|cen^{m}_{neg}\|} \tag{9}$$

$$L^{(m\to l)}_{ter} = \frac{cen^{m}_{pos} \cdot cen^{l}_{neg}}{\|cen^{m}_{pos}\| \cdot \|cen^{l}_{neg}\|} \tag{10}$$

where $L^{(m\to l)}_{ter}$ and $L^{(l\to m)}_{ter}$ represent the loss between metric to log and log to metric, respectively. By minimizing them, we learn a generalized multi-modal sample space from the clustering results and obtain modality-consistent representation. The final contrastive loss is shown in Eq. (11).

$$L_{con} = (L^{m}_{tra} + L^{l}_{tra}) + \lambda(L^{(m\to l)}_{ter} + L^{(l\to m)}_{ter}) \tag{11}$$

where λ is the hyperparameter that adjusts the weights of different modalities. It is possible to avoid widening the gap between modalities and facilitate feature fusion.

3.4. Cross-modal interaction and anomaly detection

3.4.1. Cross-modal interaction

To further analyze the deeper features of different modalities for the clustering task and perform cross-modal feature fusion, we utilize the cross-modal Transformer (Tsai et al., 2019) to bridge the gap between logs and metrics in the representation. The hidden states Z^{m}_{t}, Z^{l}_{t} are used as inputs, as shown in Fig. 6, three operations are performed: temporal convolution, positional embedding, and representation interaction.

Firstly, a 1D temporal convolutional network is used to analyze the hidden states of the metrics along the timestamps, obtaining metric representations that contain spatial–temporal relationships. Meanwhile, 1D temporal convolutional network is also utilized to analyze the logs so that the model has enough perception of different log events. Next, to prevent the timestamp offset from affecting the anomaly detection, positional embedding is carried out after convolution.

Then, we utilize multi-head cross modal attention to fuse heterogeneous features. For $m \to l$, the cross-modal attention $CM^{m\to l}$ maps metrics to query vector using a linear layer, maps log features to key vector K^{l}_{t} and value vector V^{l}_{t} and interactively learns cross modal representations, The calculation is shown in Eq. (12).

$$Att^{m\to l}_{t} = \mathrm{softmax}(\frac{Q^{m}_{t}(K^{l}_{t})^{T}}{\sqrt{d_k}})V^{l}_{t} \tag{12}$$

where $\sqrt{d_k}$ is the dimension of K^{l}_{t}. Initializing $Att^{[0]m\to l}_{t} = \tilde{Z}^{l}_{t}$, we stack D layer cross-modal attention modules and combine residual connections to achieve cross modal analysis from different angles. Thus, guided by metric, the advanced representation of logs can be generated. In this process, by assimilating the crucial information linked to the metrics, the log hidden state is consistently updated to prevent the model from becoming overly sensitive to specific modal data. Similarly, for $l \to m$, we obtain $C^{l\to m}_{t} \in \mathbb{R}^{k\times d_m}$. To facilitate cross modal interaction, we set $d_m = d_l$, and ultimately obtain multi-modal representations $C_t = concat(C^{l\to m}_{t}, C^{m\to l}_{t})$, $C_t \in \mathbb{R}^{(g+k)\times d_m}$.

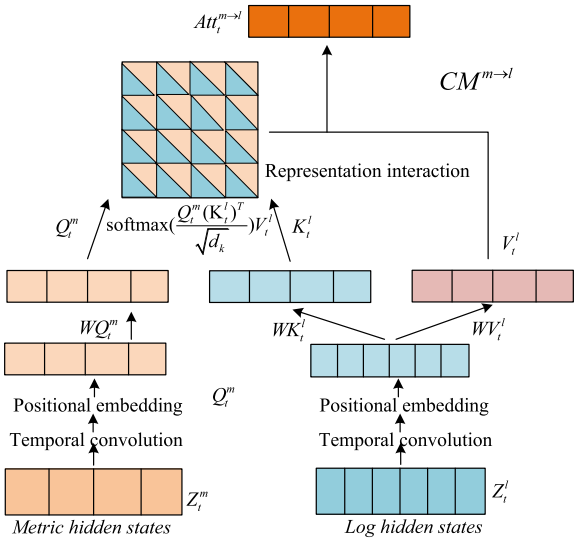


Fig. 6. An example of cross-modal interaction.

3.4.2. Anomaly detection

Based on the multi-modal representation C_t, we utilize the K-means algorithm for clustering. In order to achieve accurate clustering results and effective multi-modal representations, the clustering loss is used to simultaneously optimize the two. Specifically, MAD-CMC uses a t distribution with a degree of freedom of 1 to describe the similarity between representations C_t and cluster centers μ_j, and the calculation is as follows.

$$q_{tj} = \frac{(1 + \|C_t - \mu_j\|^2)^{-1}}{\sum_i (1 + \|C_t - \mu_i\|^2)^{-1}} \tag{13}$$

where q_{tj} can be expressed as the probability of assigning the tth sample to the jth cluster. To iteratively optimize the clustering results and obtain high confidence allocation for clustering, an auxiliary objective p_{tj} is introduced to measure the "degree" of tth samples belonging to jth clustering clusters. The calculation is shown in the Eq. (14).

$$p_{tj} = \frac{q_{tj}^2 / f_j}{\sum_i (q_{ti}^2 / f_i)} \tag{14}$$

where $f_j = \sum_t q_{tj}$. KL divergence is used to calculate the difference between P and Q. It is also optimized as a clustering loss:

$$L_{clu} = KL(P \parallel Q) = \sum_t \sum_j p_{tj} \log\left(\frac{p_{tj}}{q_{tj}}\right) \tag{15}$$

The overall loss of MAD-CMC includes two parts: multi-grained contrastive loss and clustering loss.

$$L = L_{con} + \lambda_2 L_{clu} \tag{16}$$

where λ_2 is the balance parameter.

Finally, we translate the various clustering clusters into anomaly detection results. Through the analysis of multiple sets of experiments, we determined the number of clusters to be 3. The cluster with the highest number of samples represents the normal state, followed by the abnormal state, and the cluster with the fewest samples is classified as the fuzzy state. This is because the system may experience phase fluctuations or noise factors during operation. Directly classifying them as anomalies during clustering can lead to misjudgment. To address this issue, we perform reclassification. Specifically, we calculate the spatial distance of each sample in the fuzzy cluster from the center of normal and abnormal clusters, if $\frac{1}{e^{\|C_i - cen_{(nor)}\|^2}} > \frac{1}{e^{\|C_i - cen_{(abnor)}\|^2}}$, the fuzzy sample is normal. Until all the samples of the fuzzy clusters are allocated and the anomaly detection results Y are obtained. The entire MAD-CMC learning process is shown in Algorithm 1.

4. Experiments

During the experiment, we evaluated MAD-CMC by discussing the following questions:

- How effective is MAD-CMC compared to baseline methods?
- How sensitive is MAD-CMC to various parameter settings?

Algorithm 1: The learning process of MAD-CMC

Input: Metric sequence W^m, Log sequence W^l, number of cluster I, number of epochs p
Output: Abnormal detection results: y

Pre-training;
    Update VGAE by minimizing Eq. (2) without pseudo-label
    Update AE by minimizing Eq. (3) without pseudo-label
Initialize the Encoder for VGAE and AE
for $epoch \leftarrow 1$ *to* p **do**
    $Z^m = VGAE - Encoder(W^m)$;
    $Z^l = AE - Encoder(W^l)$;
    Obtain $C^{l \to m}$ and $C^{m \to l}$ via Cross-modal Transformer;
    Integrate $C^{l \to m}$ and $C^{m \to l}$ to obtain C;
    Perform k-means on C to obtain the pseudo-label;
    Compute multi-grained contrastive learning loss L_{con};
    Compute clustering loss L_{clu};
    `// Backward propagation`
    Update the whole model by minimizing Eq. (16);
Obtain the final clustering results;
Obtain anomaly detection results based on clustering results;
return Y ;

- What is the contribution of each module in MAD-CMC?
- How robust is MAD-CMC when faced with varying levels of noise?

4.1. Experiment setting

In this section, we describe the experimental datasets, baseline models, evaluation measurements, and implementation.

4.1.1. Dataset

We evaluate the performance of MAD-CMC on two real datasets, as follows:

Generic AIOps Atlas (GAIA) dataset[1] was collected by CloudWise from a simulated microservice system environment consisting of five hosts and ten microservices, including dbservice1, dbservice2, webservice1, webservice2, and others. According to the same service names, such as dbservice1 and dbservice2, the instances of 10 services can be categorized into 5 groups. Among them, the dbservice contains 54,042 metrics and 6 million logs, while the web-service includes 128,522 metrics and 16 million logs. The remaining services, namely mobservice, redisservice and loginservice, each contain 70,000 metrics and 8 million logs. Each service involves 83 distinct metric variables, and the log templates vary between the different services.

Hades_Dataset is a real dataset[2] published by Lee, Yang, Chen, Su, Yang et al. (2023) for evaluating the effectiveness of anomaly detection methods based on metrics and logs. Researchers deployed Spark 3.0.3 on a distributed systems cluster and performed 19 different types of workloads, covering various multiple business application scenarios. It contains 61,664 metrics and 1 million logs, among them, there are 11 metric variables and 117 log templates.

Considering that this study follows an instance-based method, to further explore the effects of MAD-CMC and baseline methods on different service instances, we extract two distinct sizes of service, "dbservice" and "webservice", from the large-scale GAIA dataset for separate analysis, referred to as D1 and D2 respectively. The GAIA dataset containing all services is called D3, and the Hades_dataset is named D4. The details are shown in Table 1. Among them, Metric Length represents the number of metrics, all the metrics in the dataset are collected in seconds. Log Message is the number of logs, logs can be collected tens or hundreds per second. Metric variable is the number of variables, such as "iowait" and "Cpu_core". Log templates is the number of log templates after Drain parsing. Anomaly ratio is the proportion of abnormal data. Since the labels of the D4 are manually annotated, we further invited three graduate students in the field of Artificial Intelligence for IT Operations (AIOps) to analyze the labels of D4, and combined the results of MAD-CMC and Hades detection to modify a small number of labels in D4 to minimize the interference of external factors. Finally, 80% of each dataset was extracted for training, and the remaining 20% was used for testing.

[1] https://github.com/CloudWise-OpenSource/GAIA-DataSet.
[2] https://zenodo.org/records/7609780.

Table 1
The detail of dataset.

Dataset	Metric length	Log message	Metric variable	Log templates	Anomaly ratio
D1	54,042	6238,928	83	13	3.1%
D2	128,522	16,659,741	83	15	2.4%
D3	405,923	52,329,993	83	43	2.1%
D4	61,664	1,037,492	11	117	24.6%

4.1.2. Baseline methods

We compare MAD-CMC with several baseline models, including log-based and metric-based as well as multi-modal anomaly detection methods. Specifically, DeepLog (Du et al., 2017): the classic method based on log index; LogRobust (Zhang et al., 2019): the classic method based on log semantics; SwissLog (Li et al., 2023): the state-of-the-art baseline method based on log semantics. OmniAnomaly (Su et al., 2019): a reconstruction model combining GRU and VAE; Anomaly-Transformer (Xu et al., 2022): the state-of-the-art baseline for metric. SCWarn (Zhao et al., 2021) is an unsupervised anomaly detection method that models metrics and logs as time series. Hades (Lee, Yang, Chen, Su, Yang et al., 2023) analyzes log semantics and metric patterns simultaneously, achieving state-of-the-art results.

4.1.3. Evaluation measurements

We formalize the anomaly detection task as a classification problem, and use three commonly used measures to evaluate the model: Precision: The proportion of true anomalies among the anomalies detected by the model. $Precision = \frac{TP}{TP+FP}$. Recall: The proportion of all true anomalies that are accurately detected as anomalies by the model. $Recall = \frac{TP}{TP+FN}$. F1-score: The harmonic mean of the precision and recall. $F1 - score = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$. Where TP is the number correctly identified as abnormal samples, FP is the number of normal samples falsely trigger alarms, FN is the number of undetected anomalies.

4.1.4. Implementation

This experiment is conducted on a Linux server equipped with an NVIDIA GeForce RTX 2080 Ti GPU, which has 12 GB of graphics memory. The programming environment includes Python 3.8 and PyTorch 1.9.0+cu111.

During the experiment, we set the sliding window size to 20 and the step size to 1. For MAD-CMC, we set $o = 25$for GAIA dataset and $o = 5$ for Hades dataset to construct metric feature graph, the hidden size is 10. To facilitate cross-modal interaction, we also set the AE hidden size $d = 10$ for log. The number of clusters is set to $I = 3$. We use the Adam (Kingma & Ba, 2015) optimizer with an initial learning rate of 0.001. The batch size is 64, and the maximum training epoch is 100. In order to ensure the fairness of the comparative experiment, we reproduced all baselines according to their public source code. For the model related parameters (such as feature dimensions), we utilize the same configuration explicitly mentioned in the papers. For data-related parameters (such as window size), we adjust them based on the scope of the paper or datasets. All baselines undergo experimental analysis on the same device. Additionally, we apply the same processing to the dataset before feeding it into the model to ensure that all baseline comparisons are carried out on a consistent dataset.

4.2. Analysis of abnormal detection results

The comparison results are presented in Table 2. It can be observed that the F1-score of MAD-CMC across different datasets exceeds 0.9, indicating that MAD-CMC not only detects the majority of anomalies in the microservices system but also rarely generates false alarm alerts.

Among the log-based anomaly detection methods, semantic-based LogRobust and SwissLog outperform the index-based DeepLog. This suggests that log semantics are resilient to changes in log statements, noise, and other factors, enhancing the model's ability to detect anomalies in unstable logs. The Rec of SwissLog is higher than that of Logrobust by 6.2%, 4.4%, 4%, and 3.5% in different datasets. This is because SwissLog employs the BERT to extract more stable log semantics and time embeddings, enabling the detection of time interval anomalies. However, their performance is lower than MAD-CMC, SwissLog's F1-score is lower than 14.6%, 16.8%, 17.5%, 42.2% in different datasets. The reason is that MAD-CMC analyzes both logs and metrics, which can more comprehensively reflect the system status. Although MAD-CMC ignores time interval anomalies from log perspective, the introduction of metrics compensates for the intricate temporal relationships in the system. At the same time, we extract statistical features and capture quantitative anomalies in the logs.

Among metric-based anomaly detection methods, OmniAnomaly's Rec is 4.4%, 9.2%, 12.5%, and 49% lower than Aniomaly-Transformer on four datasets, respectively. The Anomaly-Transformer utilizes the Transformer, which offers several advantages, such as self-attention and feedforward neural network. Additionally, it incorporates a correlation discrepancy loss to quantify the disparities between normal and abnormal sequences, detecting more abnormal sequences. However, their effectiveness is not as good as MAD-CMC. This is because metric-based anomaly detection methods overly focus on metrics, and overlook other types of monitoring data that cannot fully reflect anomalies. Especially in noisy environments, a lack of knowledge about historical anomalies can lead to misjudgment of the model.

Compared to SCWarn, MAD-CMC has a higher F1-score of 38.2%, 38.2%, 40.6% and 57.4% on different services. The reasons include the following. Firstly, when modeling logs, SCWarn models them as time series, ignoring the semantic and statistical

Table 2
Experiment results on different datasets.

Models	D1			D2			D3			D4		
	Pre	Rec	F1	Pre	Rec	F1	Pre	Rec	F1	Pre	Rec	F1
DeepLog	0.426	0.719	0.535	0.411	0.682	0.512	0.364	0.681	0.474	0.218	0.409	0.284
LogRobust	0.766	0.817	0.789	0.707	0.811	0.755	0.710	0.772	0.740	0.307	0.688	0.424
SwissLog	0.790	0.879	0.832	0.764	0.855	0.806	0.759	0.812	0.784	0.361	0.723	0.481
OmniAno	0.402	0.673	0.503	0.390	0.611	0.476	0.373	0.563	0.448	0.595	0.790	0.678
AnoTrans	0.457	0.717	0.558	0.451	0.703	0.549	0.453	0.688	0.546	0.628	0.839	0.718
SCWarn	0.533	0.676	0.596	0.530	0.671	0.592	0.482	0.651	0.553	0.289	0.382	0.329
Hades	**0.972**	0.969	0.970	0.933	0.965	0.949	0.936	0.959	0.947	**0.873**	0.889	0.880
MAD-CMC	0.961	**0.997**	**0.978**	**0.963**	**0.987**	**0.974**	**0.940**	**0.979**	**0.959**	0.868	**0.942**	**0.903**

features of the logs. Secondly, when analyzing metrics, only LSTM is used to capture the temporal relationships, ignoring the spatial dependencies between different variables. Thirdly, SCWarn employs simple splicing to fuse metrics and log features, which may introduce additional noise due to the gap between different modal data. In contrast, MAD-CMC utilizes a cross-modal Transformer model to integrate metrics and logs, enabling the complementary analysis of system state using multi-modal data.

Although the F1-score of MAD-CMC in D1 is only 0.8% higher than Hades, a detailed analysis of the effects of the two models in different datasets shows that MAD-CMC has a higher Rec. In the small-scale D1 dataset, Hades can detect a large number of anomalies, but its Rec is still 2.8% lower than MAD-CMC. As we transition from D1 to D2 and D3, the number of log templates increases, and the data becomes more complex, leading to potential misjudgments by Hades. At this time, its Pre and Rec are both lower than MAD-CMC. In other microservice systems, such as the D4 dataset with fewer metric variables but more log templates, the Pre of the two models is close, but the Rec of Hades is 5.3% lower than that of MAD-CMC. Therefore, we conclude that the performance of MAD-CMC across various microservice systems is more stable and capable of detecting a greater number of anomalies. The reason is that (1) Hades employs Fast-text to analyze log semantics, but lacks analysis of statistical features. (2) Hades employs causal CNN to model the relationships between similar types of metrics and various other metrics. In contrast, MAD-CMC utilizes GNNs to analyze the dependencies between the current variable and all other variables with greater granularity. (3) MAD-CMC employs a cross-modal Transformer that facilitates deeper and longer interactions through attention mechanisms combined with temporal convolution, layer normalization, and other techniques. This model effectively analyzes more complex operational states in microservice systems. Furthermore, the fine-grained contrastive learning further optimizes the multimodal representation and detection results.

In summary, MAD-CMC achieves the best performance across various service datasets. Compared to other baseline models, the unsupervised learning approach of MAD-CMC is effective in detecting various system anomalies.

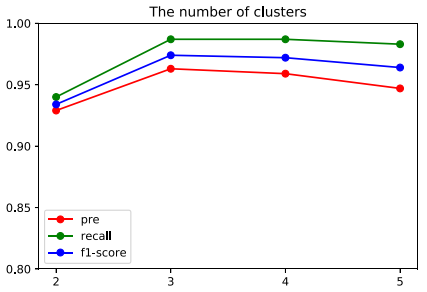
4.3. Parameter sensitivity

To further explore the impact of different parameter settings on the anomaly detection results, we conduct parameter sensitivity experiments. Specifically, We analyze the impact of different clustering numbers, sliding window sizes, and hidden feature dimensions on anomaly detection results. The experiments are conducted in the D2 dataset with moderate data volume.

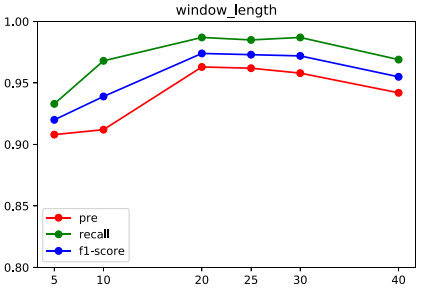
The primary objective of this study is to identify anomalies in the microservices system, which is achieved through the interpretation of the clustering results. Therefore, varying the number of clustering clusters has an impact on the results of anomaly detection. We set the number of clusters $I = 2, 3, 4, 5$ respectively. As shown in Fig. 7(a), when the number of clustering clusters is 2, the cluster with fewer samples is judged to be abnormal. It is evident that the prediction accuracy is significantly reduced at this point. This is because simply clustering monitoring data into two clusters does not align with the actual system state, and there are some indistinguishable fuzzy samples in the system, which are misclassified during clustering. When the number of clusters is greater than 2, the cluster with the highest number is judged as normal, followed by anomalies, and then the remaining clusters are separately judged. When $I = 3$, the anomaly detection effect is the best because it aligns closely with the system's operating state. Further differentiation of fuzzy clusters can significantly reduce the occurrence of false positives. When $I = 4, 5$, F1-score slightly decreases, indicating that using multiple sets of fuzzy clusters does not improve detection accuracy. On the contrary, analyzing multiple fuzzy clusters separately will increase computational complexity.

Different sliding window sizes can impact the sensitivity and accuracy of the model. Smaller windows can capture the system state more finely, but they may struggle to capture long-term abnormal behavior. As shown in Fig. 7(b), When the window size is 5, the F1-score is at its lowest because the limited information contained in a window that is too small hinders the model's ability to effectively learn the system's running state. Some anomalies need to be fully visible through a broader perspective. As the window size increases, the system's behavior in the time series is thoroughly explored. When the window size is 20, the prediction performance is optimal, indicating that the model not only captures more contextual information but also can sensitively detect abnormal behavior. When the window size exceeds 30, the performance decreases. This is because larger windows contain redundant information, making the relationships within and between modalities more complex. The model is affected by noise interference, which makes it difficult to learn distinctive features from metrics and logs.

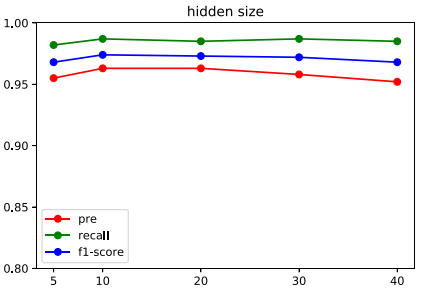
The representation capability of the model is determined by the different sizes of metrics and log hidden state dimensions, which in turn affects the results of anomaly detection. As can be observed from Fig. 7(c), different sizes of dimensions have a slight effect



(a) The number of clusters



(b) Window length



(c) Hidden size

Fig. 7. Experimental results of relevant parameters.

on the anomaly results, and the anomaly detection effect is poor when the dimension is 5. At this time, it is also necessary to adjust the number of attention heads in the cross-modal Transformer to 1. Smaller dimensions limit the model's ability to accurately model metrics and logs, resulting in the loss of useful information and difficulty in capturing complex patterns and relationships. When the dimension is 10, 20, 30, the effect is basically close, but continue to increase the dimension, the prediction effect will be reduced. The reason is that larger dimensions increase the number of parameters in the model, and although they have a richer representation, there is a risk of overfitting, reducing the model's ability to generalize to anomalous data. Considering the complexity and scale of the data, we set $d_m = d_l = 10$.

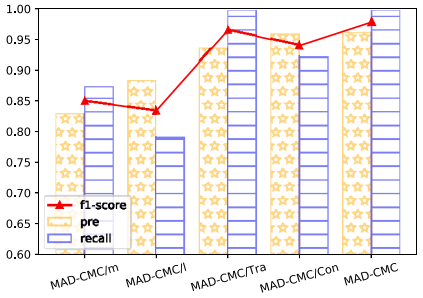
4.4. Ablation experiments

In order to further explore the effectiveness of each module of MAD-CMC, we conducted ablation experiments. (1) MAD-CMC/m represents a variant that removes metric data, MAD-CMC/l represents a variant that removes log data. (2) MAD-CMC/Con does not perform multi-grained contrastive learning. (3) MAD-CMC/Tra removes cross-modal Transformer module and adopts simple splicing to fuse the multi-modal features.

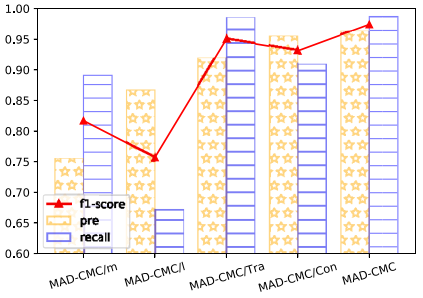
As can be seen from Fig. 8, MAD-CMC significantly outperforms MAD-CMC/m and MAD-CMC/l. Both Pre and Rec are significantly lower, which indicates that over-reliance on a certain type of monitoring data not only leads to misjudgment, but also makes it difficult to fully reflect the system state and miss some anomalies. In addition, the anomaly detection performance based on multi-modal data variant models MAD-CMC/Con and MAD-CMC/Tra is also superior to MAD-CMC/m and MAD-CMC/l, which further proves the advantages of multi-modal data.

MAD-CMC/Con is degraded in various indicators than MAD-CMC. This shows that although the hidden features in MAD-CMC can reflect the system state from different perspectives, the microservice system is complex and diverse, and even noise is introduced during the data collection process, which makes it difficult to effectively distinguish all normal and abnormal representations without the help of labeled data. And the contrast learning method designed in this paper can obtain more robust multi-modal features, which is more advantageous in dealing with certain fuzzy samples.

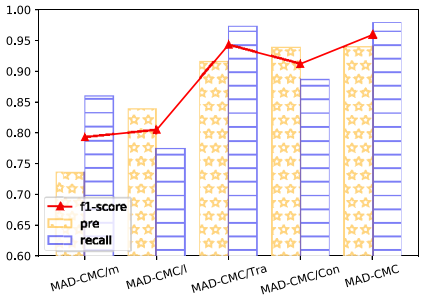
The cross-modal Transformer model used in this study demonstrates its irreplaceable role, as evidenced by the fact that MAD-CMC/Tra is lower than MAD-CMC in F1-score across different datasets. One of the reasons is that Transformer can achieved a more detailed representation by further analyzing hidden states. MAD-CMC/Con has better Pre than MAD-CMC/Tra because the multi-head cross-modal attention module can utilize information from one mode to guide the learning of features in another mode. This allows metrics and logs to complement each other, reducing false positive samples. Furthermore, benefiting from the calculation of attention scores for different metrics and log events, MAD-CMC can focus on key representations.



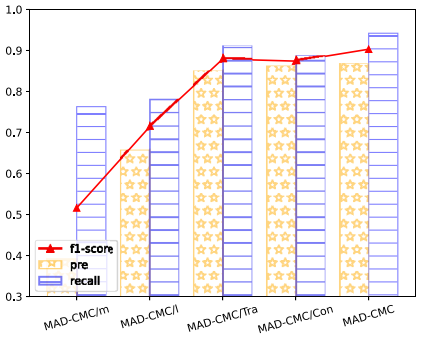
(a) Ablation experiment on D1



(b) Ablation experiment on D2



(c) Ablation experiment on D3



(d) Ablation experiment on D4

Fig. 8. Ablation experiment on various dataset.

Table 3
Experiment results on noisy datasets.

φ	D1			D2			D3			D4		
	Pre	Rec	F1	Pre	Rec	F1	Pre	Rec	F1	Pre	Rec	F1
0%	**0.961**	**0.997**	**0.978**	**0.963**	**0.987**	**0.974**	**0.940**	**0.979**	**0.959**	**0.868**	**0.942**	**0.903**
1%	0.955	0.990	0.972	0.942	0.977	0.968	0.931	0.975	0.952	0.863	0.940	0.899
5%	0.939	0.990	0.963	0.930	0.976	0.952	0.914	0.970	0.941	0.845	0.930	0.885
10%	0.908	0.977	0.941	0.920	0.979	0.948	0.907	0.971	0.937	0.841	0.926	0.881

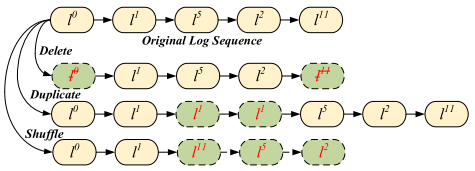
4.5. Robustness evaluation

To verify the robustness of MAD-CMC, we perform noise injection on multi-modal monitoring data. Specifically, we randomly extract ratio φ of the test set, randomly changes the metrics and logs, and replaces the original data in the test set with the generated random samples. As shown in Fig. 9, aiming to metrics, we randomly select 15 variables from datasets D1, D2, and D3, and 3 variables from dataset D4. These selected variables are then replaced with random numbers that obey any distribution at different time points. Aiming to logs, we randomly delete, duplicate, and shuffle the original log sequence.

Table 3 shows the experimental results of MAD-CMC on different datasets with different sizes φ. It can be observed that as φ increases, the Pre and Rec slightly decreased. However, even with $\varphi = 10\%$, and too many noisy samples in the dataset, all evaluation values of MAD-CMC on D1, D2, and D3 are still over 0.9, and the F1-score on D4 only drops by 0.22, which proves its robustness. In addition, when $\varphi = 10\%$, the Rec decreases by 2%, 0.8%, 0.8%, and 1.6% in the four datasets, respectively. But the Pre decreases by 5.3%, 4.3%, 3.3% and 2.7%. This shows that MAD-CMC will recognize some of the noise samples as anomalies, which is realistic. During system operation, the model identifies significant fluctuations in noise data as abnormal, aiding in troubleshooting. However, mistaking abnormal noise for normal sounds can lead to greater losses. Furthermore, even in noisy data, MAD-CMC still outperforms the SCWarn, which further proves the advantage of the contrastive multi-modal representation clustering.

Timestamp	m^1	m^2	...	m^i	...	m^j	...	m^k
...	...	...	...	...	...	...	...	...
t_i	0.061956	2.406842e+15	...	1.031007 ~~0.000142~~	...	1.251344	...	0.047389
...	...	...	...	...	...	...	...	...
t_j	0.061949	0.068714 ~~1.132150e+15~~	...	0.000000	...	0.249167 ~~1.249760~~	...	0.000211 ~~0.867986~~
t_{j+1}	0.099191 ~~0.062644~~	1.132178e+15	...	0.000000	...	1.712566 ~~1.249218~~	...	3.726608 ~~0.604898~~
...	...	...	...	...	...	...	...	...

(a) Noise samples on metrics



(b) Noise samples on logs

Fig. 9. Noise samples on metric and log.

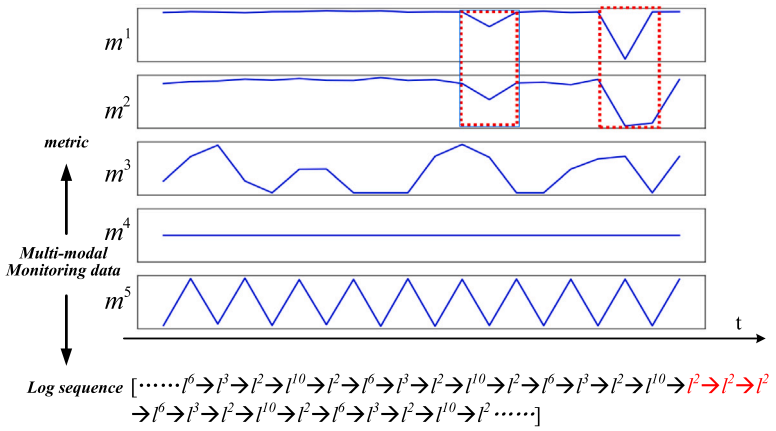


Fig. 10. Multimodal case study.

4.6. Case study

In this section, we select a representative case from the D2 dataset to examine the effectiveness of MAD-CMC. Although the dataset comprises 83 variables, for the sake of clarity, we focus on 5 representative metric variables within a window, along with the corresponding partial log sequence fragments.

In Fig. 10, the horizontal axis represents time, and the red dotted box represents the true anomaly. We observe that m^1 and m^2 exhibit similar trends over time, indicating a correlation between them. In the red dotted box, m^1 and m^2 exhibit different changes and are no longer consistent, which indicates an anomaly may occur at this time. The unimodal OmniAnomaly may be regarded as normal since other related metrics continue to fluctuate within the normal range. In contrast, MAD-CMC models the correlation between variables, thereby accurately detecting anomalies. Furthermore, even if the model disregards the correlation, both MAD-CMC and Hades can still identify abnormalities by analyzing the log data, which is not accessible in the single-modal algorithms.

In the corresponding log sequence, we find that $l^6 \rightarrow l^3 \rightarrow l^2 \rightarrow l^{10} \rightarrow l^2$ is repeatedly executed in the current time window, which represents a normal business activity of the system. At a certain moment, the l^2 is repeated three times. which is different from the normal sequence. The unimodal LogRobust struggles to detect anomalies when only analyzing log semantics, as each event semantics in the sequence is normal. MAD-CMC can capture this quantitative anomaly by analyzing its statistical relationship. Although SwissLog can also detect abnormal changes by analyzing time intervals, if the duration of l^2 is brief, the model may interpret it as noise and make an incorrect judgment. Besides, if the model can further analyze the metric data, it will obtain more abnormal features, making it easier to judge the current system status. Therefore, compared with the single-modal algorithm, MAD-CMC can capture more subtle anomalies.

5. Discussion

Theoretical implications. Firstly, most studies focused on using single type of monitoring data to detect system anomalies. For logs, they described system states by extracting semantic (Zhang et al., 2019) and temporal embeddings (Fu et al., 2023) from logs, but logs cannot record fine-grained status, such as "memory_usage". For metrics, existing studies not only analyzed their temporal relationships (Su et al., 2019) but also explored the relationships between different variables (Shi et al., 2023).

While it can sensitively capture system changes, this sensitivity can sometimes cause the model to make inaccurate judgments about acceptable fluctuations. SCWarn (Zhao et al., 2021) and DAM (Chen et al., 2022) have analyzed these two types of data simultaneously. However, there were representation differences between different modal data, and simple feature concatenation cannot fully leverage the advantages of various modal data. Therefore, Hades (Lee, Yang, Chen, Su, Yang et al., 2023) and UAC-AD (Liu et al., 2024) adopt cross-modal attention mechanism for metric-log interaction, achieving a state-of-the-art effect. Based on this, we introduce a novel cross-modal Transformer to perform interactions of metrics and logs. In addition, MAD-CMC proposes a contrastive learning approach, which further explores the consistency of across modal representation and improves the model's discriminative ability.

Secondly, the unsupervised training mode avoids the dependence of MAD-CMC on labels. Existing studies usually fall into two categories. One category involves training the model using all or part labeled data to learn discriminative normal and abnormal representations (Lee, Yang, Chen, Su, Yang et al., 2023; Zhang et al., 2019). However, annotating a large amount of multimodal data is time-consuming and difficult. Another category focuses on learning normal state representations and identifying samples that deviate from the normal state as abnormal (Du et al., 2017). This practically implies the utilization of labels, and ensuring that the model learns only normal representations is challenging due to objective factors such as noise. In contrast, MAD-CM is trained by pre-training and fine-tuning, and clustered by K-means algorithm. The entire process does not require labels, and the monitoring data is directly divided into different category clusters. Therefore, MAD-CMC can provide accurate detection results for microservice systems that lack sufficient labeled samples.

Third, concerning the efficiency of MAD-CMC, we employ a hierarchical architecture that enables the optimization of different modal data separately. This parallel approach significantly reduces the overall computation time. In terms of models, MAD-CMC utilizes AE modeling logs and VGAE modeling metrics, both of which have low computational complexity, and are able to run efficiently in most real-world application scenarios, thereby fulfilling the requirements for real-time processing. Regarding pre-training and fine-tuning training method, the pre-training model can significantly reduce the training time, and fine-tuning of the target task can improve the efficiency of the model in specific scenarios. From a data perspective, the comparison results with the baseline model show that MAD-CMC outperforms the metrics-based approach by over 19% in the F1-score, exceeds the log-based method by more than 18%, and surpasses SCWarn by 38%. The ablation experiment results show that any modality data is crucial for MAD-CMC. When logs are removed, Rec decreases by more than 20% on various datasets. In the robustness study, when the data noise injection rate reaches 10%, the F1-score of MAD-CMC is still better than that of the SCWarn. In summary, the efficient combination of various modules of MAD-CMC improves the efficiency of anomaly detection in microservice systems.

Practical application and requirements. In practical applications, whether MAD-CMC can be applied to microservice systems in different scenarios. Although we experimentally validated it in two publicly dataset, it is not yet known that our method can be generalized to all other systems. Considering this, we employ datasets from different scenarios, and the fault types and workloads are representative. The experimental results show that MAD-CMC outperforms existing baseline methods in both datasets. Moreover, we conduct a robustness analysis, and even in some large and complex industrial environments where data may be affected by noise and missing values, MAD-CMC still maintains stable performance. In the future, we will evaluate MAD-CMC on more datasets.

MAD-CMC adopts a hierarchical architecture, how effective is this for microservice systems of different scales? Considering this, MAD-CMC allows flexible model simplification. Specifically, a simplified metric graph can be constructed based on the system structure, and the SVD algorithm can further reduce the dimensionality of log semantics. During the fine-tuning process, the cross-modal Transformer structure can be adjusted to reduce the number of internal modules. It is even possible to adjust the Transformer structure to a cross-modal attention, retaining interactive functionality while significantly simplifying the model, thereby extending the applicability of MAD-CMC.

A special requirement comes from data alignment. MAD-CMC analyzes the status of microservice systems based on multimodal data, which requires ensuring that the metrics and logs in each window come from the same service instance and are aligned by timestamp, so as to avoid misclassification caused by data from different sources carrying different system information. Another requirement that affects the efficiency of anomaly detection comes from pre-training. Pre-training can better initialize the model parameters, making the model converge more quickly and improve the clustering effect in the fine-tuning process. Therefore, the dataset size has an impact on the pre-training, and under the same conditions, a larger dataset can ensure that the model learns more key features during pre-training and reduces the risk of overfitting.

6. Conclusions

In this work, we propose an unsupervised anomaly detection method for microservice system, called MAD-CMC, which jointly analyzes metrics and logs within a framework. It firstly captures spatial–temporal correlation in metrics as well as semantic and statistical features of logs. Next, a cross-modal Transformer model is employed to facilitate metric-log interactions and learn multimodal representation. Finally, we utilize the K-means algorithm for clustering and propose a multi-grained contrastive learning method. Guided by the clustering results, multimodal representation is optimized from both the intra- and inter-modal aspects, while the optimized representation can also improve the clustering results. In addition, in order to further enhance discriminability of model, we design a dynamic weighting formula, which compel the model to pay more attention to abnormal samples. By training in this way, we avoid the reliance on labeled data. Experimental results on real-world datasets show that MAD-CMC outperforms the state-of-art methods. The results of ablation experiments show that the Rec of MAD-CMC is at least 10% better than that of single modal data. In addition, the MAD-CMC maintains an F1-score of over 0.9 even in the presence of varying degrees of noise in the data.

In practical applications, operator feedback can provide valuable input to the model, which is currently lacking in MAD-CMC. In future research, we will implement a feedback mechanism that can be triggered periodically to correct instances of misjudgment. Besides, our experiment revealed that the convergence speed of metrics and logs is inconsistent, and MAD-CMC treats them equally. In the future, we will introduce a modal balance strategy to suppress high-yield modes while enhancing low-yield modes.

CRediT authorship contribution statement

Peipeng Wang: Writing – review & editing, Writing – original draft, Validation, Methodology, Investigation, Conceptualization. **Xiuguo Zhang:** Supervision, Methodology, Funding acquisition, Conceptualization. **Yutian Chen:** Validation, Methodology. **Zhiying Cao:** Supervision, Investigation, Funding acquisition.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (Grant No. 52231014), Liaoning Province Applied Basic Research Program Project, China (Grant No. 2023JH2/101300195) and the Cultivation Program for the Excellent Doctoral Dissertation of Dalian Maritime University, China.

Data availability

Data will be made available on request.

References

Chen, Y., Yan, M., Yang, D., Zhang, X., & Wang, Z. (2022). Deep attentive anomaly detection for microservice systems with multimodal time-series data. In *2022 IEEE international conference on web services* ICWS, (pp. 373–378). http://dx.doi.org/10.1109/ICWS55610.2022.00062.

Devlin, J., Chang, M.-W., Lee, K., & Kristina, T. (2019). Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies* NAACL-HLT, (pp. 4171–4186). http://dx.doi.org/10.18653/v1/n19-1423.

Ding, C., Sun, S., & Zhao, J. (2022). Mst-gat: A multimodal spatial–temporal graph attention network for time series anomaly detection. *Information Fusion*, *89*, 527–536. http://dx.doi.org/10.1016/j.inffus.2022.08.011.

Ding, H., Trajcevski, G., Scheuermann, P., Wang, X., & Keogh, E. (2022). Deepsyslog: Deep anomaly detection on syslog using sentence embedding and metadata. *IEEE Transactions on Information Forensics and Security*, *17*, 3051–3061. http://dx.doi.org/10.1109/TIFS.2022.3201379.

Du, M., Li, F., Zheng, G., & Srikumar, V. (2017). Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security* CCS '17, (pp. 1285–1298). New York, NY, USA: Association for Computing Machinery, http://dx.doi.org/10.1145/3133956.3134015.

Duan, X., Ying, S., Yuan, W., Cheng, H., & Yin, X. (2021). Qllog: A log anomaly detection method based on q-learning algorithm. *Information Processing & Management*, *58*(3), Article 102540. http://dx.doi.org/10.1016/j.ipm.2021.102540.

Fu, Y., Liang, K., & Xu, J. (2023). Mlog: Mogrifier lstm-based log anomaly detection approach using semantic representation. *IEEE Transactions on Services Computing*, *16*(5), 3537–3549. http://dx.doi.org/10.1109/TSC.2023.3289488.

Ge, N., Weng, X., & Yang, Q. (2014). Unsupervised anomaly detection via two-dimensional singular value decomposition and subspace reconstruction for multivariate time series. *Applied Intelligence: The International Journal of Artificial Intelligence, Neural Networks, and Complex Problem-Solving Technologies*, *53*(13), 16813–16829. http://dx.doi.org/10.1007/s10489-022-04337-y.

Hamooni, H., Debnath, B., Xu, J., Zhang, H., Jiang, G., & Mueen, A. (2016). Logmine: Fast pattern recognition for log analytics. In *Proceedings of the 25th ACM international on conference on information and knowledge management* CIKM '16, (pp. 1573–1582). New York, NY, USA: Association for Computing Machinery, http://dx.doi.org/10.1145/2983323.2983358.

He, S., He, P., Chen, Z., Yang, T., Su, Y., & Lyu, M. R. (2021). A survey on automated log analysis for reliability engineering. *ACM Computing Surveys*, *54*(6), http://dx.doi.org/10.1145/3460345.

He, P., Zhu, J., Zheng, Z., & Lyu, M. R. (2017). Drain: An online log parsing approach with fixed depth tree. In *2017 IEEE international conference on web services* ICWS, (pp. 33–40). http://dx.doi.org/10.1109/ICWS.2017.13.

Hu, S., Zhang, C., Zou, G., Lou, Z., & Ye, Y. (2024). Deep multiview clustering by pseudo-label guided contrastive learning and dual correlation learning. *IEEE Transactions on Neural Networks and Learning Systems*, 1–13. http://dx.doi.org/10.1109/TNNLS.2024.3354731.

Hu, S., Zou, G., Zhang, C., Lou, Z., Geng, R., & Ye, Y. (2023). Joint contrastive triple-learning for deep multi-view clustering. *Information Processing & Management*, *60*(3), http://dx.doi.org/10.1016/j.ipm.2023.103284.

Huang, J., Yang, Y., Yu, H., Li, J., & Zheng, X. (2024). Twin graph-based anomaly detection via attentive multi-modal learning for microservice system. In *Proceedings of the 38th IEEE/ACM international conference on automated software engineering, ASE '23* (pp. 66–78). IEEE Press, http://dx.doi.org/10.1109/ASE56229.2023.00138.

Hundman, K., Constantinou, V., Laporte, C., Colwell, I., & Soderstrom, T. (2018). Detecting spacecraft anomalies using lstms and nonparametric dynamic thresholding. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining* KDD '18, (pp. 387–395). New York, NY, USA: Association for Computing Machinery, http://dx.doi.org/10.1145/3219819.3219845.

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In *3rd international conference on learning representations, ICLR*.

Kipf, T. N., & Welling, M. (2016). Variational graph auto-encoders. arXiv preprint arXiv:1611.07308.

Lee, C., Yang, T., Chen, Z., Su, Y., & Lyu, M. R. (2023). Eadro: An end-to-end troubleshooting framework for microservices on multi-source data. In *Proceedings of the 45th international conference on software engineering, ICSE '23* (pp. 1750–1762). IEEE Press, http://dx.doi.org/10.1109/ICSE48619.2023.00150.

Lee, C., Yang, T., Chen, Z., Su, Y., Yang, Y., & Lyu, M. R. (2023). Heterogeneous anomaly detection for software systems via semi-supervised cross-modal attention. In *IEEE/ACM 45th international conference on software engineering* ICSE, (pp. 1724–1736). IEEE/ACM, http://dx.doi.org/10.1109/ICSE48619.2023.00148.

Li, X., Chen, P., Jing, L., He, Z., & Yu, G. (2023). Swisslog: Robust anomaly detection and localization for interleaved unstructured logs. *IEEE Transactions on Dependable and Secure*, *20*(4), 2762–2780. http://dx.doi.org/10.1109/TDSC.2022.3162857.

Liu, H., Huang, X., Jia, M., Jia, T., Han, J., Li, Y., et al. (2024). Uac-ad: Unsupervised adversarial contrastive learning for anomaly detection on multi-modal data in microservice systems. *IEEE Transactions on Services Computing*, 1–14. http://dx.doi.org/10.1109/TSC.2024.3411481.

Liu, Y., Yang, X., Zhou, S., Liu, X., Wang, Z., Liang, K., et al. (2023). Hard sample aware network for contrastive deep graph clustering. *vol. 37*, In *Proceedings of the AAAI conference on artificial intelligence* (pp. 8914–8922). http://dx.doi.org/10.1609/aaai.v37i7.26071, (7).

Makanju, A. A., Zincir-Heywood, A. N., & Milios, E. E. (2009). Clustering event logs using iterative partitioning. In *Proceedings of the 15th ACM SIGKDD international conference on knowledge discovery and data mining, KDD '09* (pp. 1255–1264). New York, NY, USA: Association for Computing Machinery, http://dx.doi.org/10.1145/1557019.1557154.

Meng, W., Liu, Y., Zhu, Y., Zhang, S., Pei, D., Liu, Y., et al. (2019). Loganomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs. In *Proceedings of the twenty-eighth international joint conference on artificial intelligence, IJCAI* (pp. 4739–4745). http://dx.doi.org/10.24963/ijcai.2019/658.

Oord, A. v. d., Li, Y., & Vinyals, O. (2018). Representation learning with contrastive predictive coding. arXiv preprint arXiv:1807.03748.

Runge, J., Nowack, P., Kretschmer, M., Flaxman, S., & Sejdinovic, D. (2019). Detecting and quantifying causal associations in large nonlinear time series datasets. *Science Advances*, *5*(11), Article eaau4996. http://dx.doi.org/10.1126/sciadv.aau4996.

Shi, Y., Wang, B., Yu, Y., Tang, X., Huang, C., & Dong, J. (2023). Robust anomaly detection for multivariate time series through temporal gcns and attention-based vae. *Knowledge-Based Systems*, *275*(110725), http://dx.doi.org/10.1016/j.knosys.2023.110725.

Srivastava, N., Mansimov, E., & Salakhutdinov, R. (2015). Unsupervised learning of video representations using lstms. In *Proceedings of the 32nd international conference on international conference on machine learning - volume 37* iCML'15, (pp. 843–852). JMLR.org.

Su, Y., Zhao, Y., Niu, C., Liu, R., Sun, W., & Pei, D. (2019). Robust anomaly detection for multivariate time series through stochastic recurrent neural network. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining* KDD '19, (pp. 2828–2837). New York, NY, USA: Association for Computing Machinery, http://dx.doi.org/10.1145/3292500.3330672.

Tsai, Y.-H. H., Bai, S., Liang, P. P., Kolter, J. Z., Morency, L.-P., & Salakhutdinov, R. (2019). Multimodal transformer for unaligned multimodal language sequences. In *Proceedings of the conference. association for computational linguistics. meeting, 2019* NIH public access, (p. 6558). http://dx.doi.org/10.18653/v1/p19-1656.

Wang, P., Zhang, X., Cao, Z., Xu, W., & Li, W. (2024). Loggt: Cross-system log anomaly detection via heterogeneous graph feature and transfer learning. *Expert Systems with Applications*, *251*, Article 124082. http://dx.doi.org/10.1016/j.eswa.2024.124082.

Xia, W., Wang, T., Gao, Q., Yang, M., & Gao, X. (2023). Graph embedding contrastive multi-modal representation learning for clustering. *IEEE Transactions on Image Processing*, *32*, 1170–1183. http://dx.doi.org/10.1109/TIP.2023.3240863.

Xie, Y., & Yang, K. (2024). Log anomaly detection by adversarial autoencoders with graph feature fusion. *IEEE Transactions on Reliability*, *73*(1), 637–649. http://dx.doi.org/10.1109/TR.2023.3305376.

Xu, J., Wu, H., Wang, J., & Long, Mingsheng (2022). Anomaly transformer: Time series anomaly detection with association discrepancy. In *The tenth international conference on learning representations, ICLR* (pp. 6894–6910). http://dx.doi.org/10.48550/arXiv.2110.02642.

Yan, L., Luo, C., & Shao, R. (2023). Discrete log anomaly detection: A novel time-aware graph-based link prediction approach. *Information Sciences*, *647*, http://dx.doi.org/10.1016/j.ins.2023.119576.

Yang, X., Liu, Y., Zhou, S., Wang, S., Tu, W., Zheng, Q., et al. (2023). Cluster-guided contrastive graph clustering network. *vol. 37*, In *Proceedings of the AAAI conference on artificial intelligence* (pp. 10834–10842). AAAI Press, http://dx.doi.org/10.1609/AAAI.V37I9.26285.

Zhang, S., Jin, P., Lin, Z., Sun, Y., Zhang, B., Xia, S., et al. (2023). Robust failure diagnosis of microservice system through multimodal data. *IEEE Transactions on Services Computing*, *16*(6), 3851–3864. http://dx.doi.org/10.1109/TSC.2023.3290018.

Zhang, X., Xu, Y., Lin, Q., Qiao, B., Zhang, H., Dang, Y., et al. (2019). Robust log-based anomaly detection on unstable log data. In *Proceedings of the 2019 27th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering, ESEC/FSE 2019* (pp. 807–817). New York, NY, USA: Association for Computing Machinery, http://dx.doi.org/10.1145/3338906.3338931.

Zhao, N., Chen, J., Yu, Z., Wang, H., Li, J., Qiu, B., et al. (2021). Identifying bad software changes via multimodal anomaly detection for online service systems. In *Proceedings of the 29th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering, ESEC/FSE 2021* (pp. 527–539). New York, NY, USA: Association for Computing Machinery, http://dx.doi.org/10.1145/3468264.3468543.

Zhao, N., Han, B., Li, R., Su, J., & Zhou, C. (2023). A multivariate kpis anomaly detection framework with dynamic balancing loss training. *IEEE Transactions on Network and Service Management*, *20*(2), 1418–1429. http://dx.doi.org/10.1109/TNSM.2022.3224803.

Zhao, C., Ma, M., Zhong, Z., Zhang, S., Tan, Z., Xiong, X., et al. (2023). Robust multimodal failure detection for microservice systems. In *Proceedings of the 29th ACM SIGKDD conference on knowledge discovery and data mining* KDD '23, (pp. 5639–5649). New York, NY, USA: Association for Computing Machinery, http://dx.doi.org/10.1145/3580305.3599902.

Zhao, H., Wang, Y., Duan, J., Huang, C., Cao, D., Tong, Y., et al. (2020). Multivariate time-series anomaly detection via graph attention network. In *2020 IEEE international conference on data mining* ICDM, (pp. 841–850). http://dx.doi.org/10.1109/ICDM50108.2020.00093.

Zhou, X., Peng, X., Xie, T., Sun, J., Ji, C., Li, W., et al. (2021). Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study. *IEEE Transactions on Software Engineering*, *47*(2), 243–260. http://dx.doi.org/10.1109/TSE.2018.2887384.

Zhu, J., He, S., Liu, J., He, P., Xie, Q., Zheng, Z., et al. (2019). Tools and benchmarks for automated log parsing. In *2019 IEEE/ACM 41st international conference on software engineering: software engineering in practice (ICSE-sEIP)* (pp. 121–130). http://dx.doi.org/10.1109/ICSE-SEIP.2019.00021.